

SR Proposal: Beyond Code Correctness: Hierarchical Verifier Synthesis for Autonomous Model Development

Author(s): Rob Tang Jiefeng Chen Jinsung Yoon

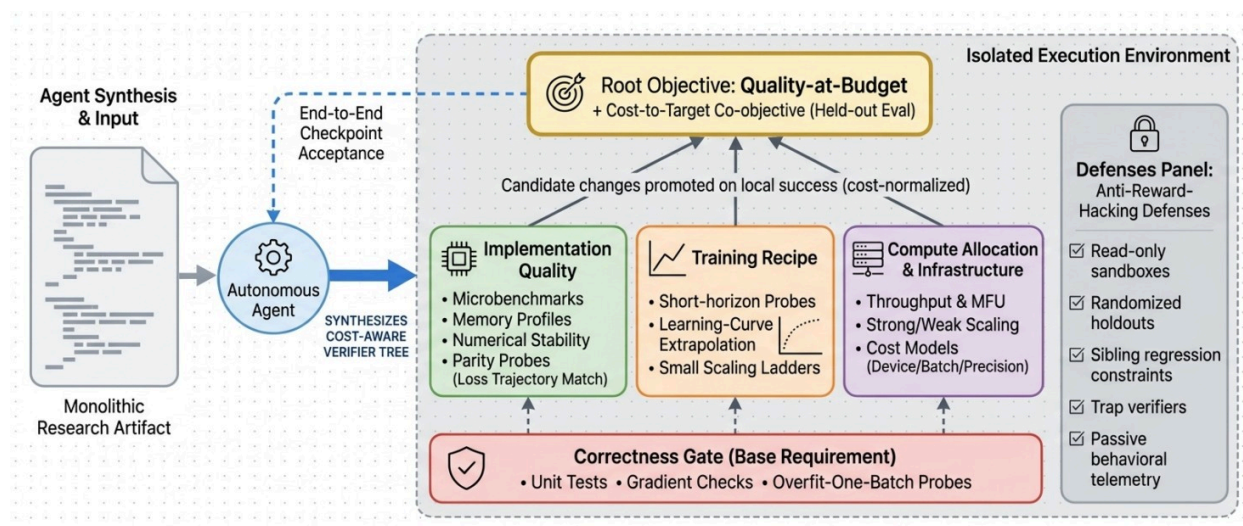
Date: Jul 31, 2026

SR candidates:

<https://hydrapse.github.io/> Gangda Deng

Project Overview

Autonomous AI/ML research agents today are built and judged around code correctness: the program runs, tests pass, a score comes out. Correctness is necessary but it is a low bar. Hand 10 teams the same correct BERT implementation and a fixed compute budget, and they will return models several points apart on GLUE. The gap comes from decisions that correctness checks never observe: which data and in what order, how training is scheduled and staged, and how compute is spent. In human practice this is the difference between an engineer who can implement a model and a researcher who can train one well.



We propose to evaluate and improve autonomous model development along 3 axes that sit above binary correctness:

- Implementation quality. Among correct implementations, throughput, memory footprint, and numerical stability vary by orders of magnitude. Naive attention and a fused kernel are both correct; only one is usable at scale.
- Training recipe. With code held fixed, final quality is determined by data selection and mixture, curriculum, optimizer and schedule, staging (pretrain, anneal, fine-tune), and hyperparameters. This is where most tacit ML expertise lives, and it is the least verified part of the pipeline.
- Compute allocation and infrastructure. The same recipe can run on 1 GPU for 100 days or on 64 GPUs for 2 days with a scaled batch size and returned learning rate, under different frameworks (FSDP, DeepSpeed, Megatron, JAX) and parallelism strategies. Time-to-result, cost-to-result, and hardware utilization (MFU) are first-class outcomes, not afterthoughts.

Method

We restructure evaluation as a cost-aware verifier hierarchy whose root is quality-at-budget on a held-out eval, with cost-to-target as a co-objective. An autonomous agent synthesizes the tree directly from the research artifact, organized by axis:

- Correctness gate: unit tests, gradient checks, overfit-one-batch probes.
- Implementation: microbenchmarks, memory profiles, numerical stability probes, and parity probes: fixed-seed short runs whose loss trajectories must match a reference within tolerance, certifying that an optimization preserves training semantics.
- Recipe: short-horizon training probes with learning-curve extrapolation, and small scaling ladders (train at 3 reduced scales, extrapolate) that act as cheap judges predicting full-run outcomes before they are paid for.
- Compute allocation: throughput and MFU measurement, strong and weak scaling tests, and simple cost models over device count, batch size, and precision.

Candidate changes are promoted only on local success normalized by cost, and accepted only if they improve the root objective at scheduled end-to-end checkpoints. Anti-reward-hacking defenses carry over from our prior verifier work: read-only sandboxes, randomized holdouts, sibling regression constraints, trap verifiers, and behavioral telemetry used for passive auditing only, never as reward.

Benchmark and harness

No existing benchmark isolates these axes. MLE-bench scores a final leaderboard metric and conflates every source of skill; RE-Bench contains a few isolated efficiency tasks; AlgoPerf fixes the

code and varies only the training algorithm. We will build a compact task suite and an open-source harness. Each task provides reference architecture code (e.g., PyTorch), a data pool, a compute budget, and a held-out eval; the agent must deliver the highest-performing model within budget while ensuring both functional correctness and execution efficiency (e.g., via JAX JIT compilation and memory optimizations). Tasks come from reproducible small-scale settings with strong public human baselines: a scaled NanoGPT speedrun (GPT-2 124M/350M class, scaled to multi-hour training budgets on 8 H100s to test compute allocation efficiency), academic-budget BERT pretraining with GLUE evaluation, and a semantics-preserving throughput optimization task. Perturbed variants (modified architectures, tokenizers, data distributions, memory caps) prevent agents from winning by retrieving public speedrun recipes.

Research Questions

RQ1, measurement: Which cheap verifiers actually predict final model quality and training cost? We calibrate every verifier on every axis against ground-truth end-to-end outcomes, producing the first predictiveness map for cheap training signals.

RQ2, synthesis: Can an agent reliably synthesize a useful multi-axis verifier hierarchy from a monolithic research artifact?

RQ3 efficiency: Under a strictly fixed compute budget, how does hierarchy-guided search compare against flat best-of- n search (AIDE-style) and direct end-to-end iteration in terms of:

1. Quality-at-Budget: Achieving higher verified candidate quality within a fixed token/time budget?
2. Cost-to-Target: Reaching a benchmark quality threshold with lower total compute expenditure?

Plan (16 weeks)

Weeks 1-4: Harness v0 and task suite. Sandboxed execution reusing the OpenHands runtime, a GPU-hour budget ledger, a verifier registry, a run database, and a leaderboard. 4 tasks at speedrun scale so a full end-to-end run costs 0.1 to 8 GPU-hours.

Weeks 5-8: Verifier synthesis and calibration. Implement per-axis synthesis and run the RQ1 calibration study. This study stands alone as a contribution regardless of downstream agent results.

Weeks 9-12: Agent experiments. Fixed budget per task per method (about 200 GPU-hours), 3 policies: end-to-end only, flat, hierarchical. All defenses on, full audit logs kept.

Weeks 13-16: Ablations and release. Remove axes, remove hierarchy, remove extrapolation judges; test generalization on perturbed tasks; writeup and open-source release.

Feasibility

The design choice that makes 4 months workable is task scale. Every task is small enough that hundreds of verified end-to-end runs fit in a modest budget, and the search problem stays real because public speedrun records show large headroom between naive and expert execution at exactly this scale. Estimated total compute: 5,000 to 10,000 H100-hours or TPU equivalent. The harness builds on infrastructure the team has already shipped (OpenHands, prior agent-benchmark harnesses), so a working v0 in 4 weeks is realistic.

Risks and Mitigations

Learning-curve extrapolation can be weakly predictive for changes that act late in training, such as decay-phase effects. Mitigation: scaling ladders and checkpoint-anchored probes, and RQ1 quantifies exactly where cheap judges fail, which is itself a finding the field lacks. Efficiency metrics invite hacking, for example inflating throughput by silently dropping work. Mitigation: parity probes gate every infrastructure change. Public recipes may leak into agents. Mitigation: perturbed held-out variants, plus a retrieval-only baseline that separates search ability from recall.

Deliverables and Impact

- Open-Source Evaluation Harness & Task Suite: A multi-axis task suite and harness for evaluating autonomous model development, featuring public speedrun records as human reference baselines.
- Verifier Synthesis Agent & Calibration Analysis: A trained verifier-synthesis agent and a comprehensive calibration report evaluating cheap-verifier predictiveness and fidelity (RQ1).
- Budget-Controlled Search Analysis: A rigorous, controlled empirical comparison of hierarchical, flat, and end-to-end-only search under fixed compute budgets (RQ3), backed by full audit and trace logs.
- Academic Publication & Open Artifacts: A conference submission (ICML 2027 or NeurIPS 2027, depending on timeline), along with open-sourced verifier trees and scoped task environments for training future research agents.

- Synergy with CAIR's AI Scientist (ScientistOne):
 - Compute-Efficient Inner Loop:** Enhances *ScientistOne*'s experimentation module by using hierarchical verifiers as fast, cheap execution gates (e.g., `torch2jax` parity and JIT checks) to eliminate invalid candidates before high-cost runs.
 - Chain-of-Evidence (CoE) Reinforcement: Provides dense audit logs that feed directly into *ScientistOne*'s verification modules, ensuring method-code alignment and preventing reward hacking in autonomous discovery.
- Dense-Reward Training Ground: Delivers verifier trees and scoped environments to serve as benchmark tasks for next-generation research agents (Scientist1.5 / ScientistTwo). Claims about AI systems that do AI research currently rest on single conflated numbers. This project supplies the measurement instrument those claims need, and turns that instrument into the training signal for the next generation of research agents.